Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Introduction

Using Sensors with Arduino

1. Introduction to Sensors

Sensors are essential components in robotics and electronics that allow devices like Arduino to interact with the physical world. Sensors can detect various environmental parameters, such as temperature, light, distance, humidity, and more. They provide real-time data that the Arduino can process to trigger specific actions or decisions in a project.

Types of Sensors:

Temperature Sensors:

LM35 and DHT11 are common temperature sensors.

The LM35 provides accurate temperature readings in Celsius, while the DHT11 can read both temperature and humidity.

Light Sensors:

Photoresistors (LDR) measure the intensity of light. They can be used in projects like automatic night lamps or solar trackers.

Distance Sensors:

Ultrasonic sensors (like the HC-SR04) use sound waves to measure the distance between the sensor and an object.

Infrared (IR) sensors can detect objects by measuring infrared light reflections.

How to Interface Sensors with Arduino:

To use sensors with Arduino, you need to understand the type of output they provide. Sensors can output either digital or analog signals:

Digital Sensors:

These sensors output either HIGH (on) or LOW (off) signals. For example, an IR sensor can detect the presence of an object and send a HIGH signal to the Arduino.

Analog Sensors:

Analog sensors output a range of values. For instance, a temperature sensor may provide varying voltages based on the temperature, which the Arduino converts into a readable value.

To interface a sensor with Arduino:

Connect the sensor to the appropriate Arduino pins (digital or analog).

Write a program (or sketch) that reads the sensor data.

Use the data to control other components or display it on a screen.

2. Reading Sensor Data

Once a sensor is connected to the Arduino, we can read the sensor's output using Arduino's built-in functions, such as analogRead() or digitalRead(). The data can be displayed in the Serial Monitor, a feature of the Arduino IDE that allows real-time data output.

Displaying Data on the Serial Monitor:

The Serial Monitor allows you to see the values your Arduino is reading from the sensor. Here's how to read sensor data and display it:

Begin Serial Communication:

Use Serial.begin(9600); in the setup() function to start communication with the Serial Monitor.

Reading Data:

Use analogRead() for analog sensors or digitalRead() for digital sensors.

For example, to read temperature from an LM35 sensor, use:

```cpp
int sensorValue = analogRead(A0); // Reading from analog pin A0
```

3.Print the Data:

- Use Serial.print() or Serial.println() to display the data on the Serial Monitor.
- Example:

```cpp
Serial.print("Temperature: ");
Serial.println(sensorValue); // Print the temperature value
```

---

3. Hands-On Activity: Interface a Temperature Sensor with Arduino

In this hands-on activity, you'll connect a temperature sensor to the Arduino and display the temperature readings on the Serial Monitor.

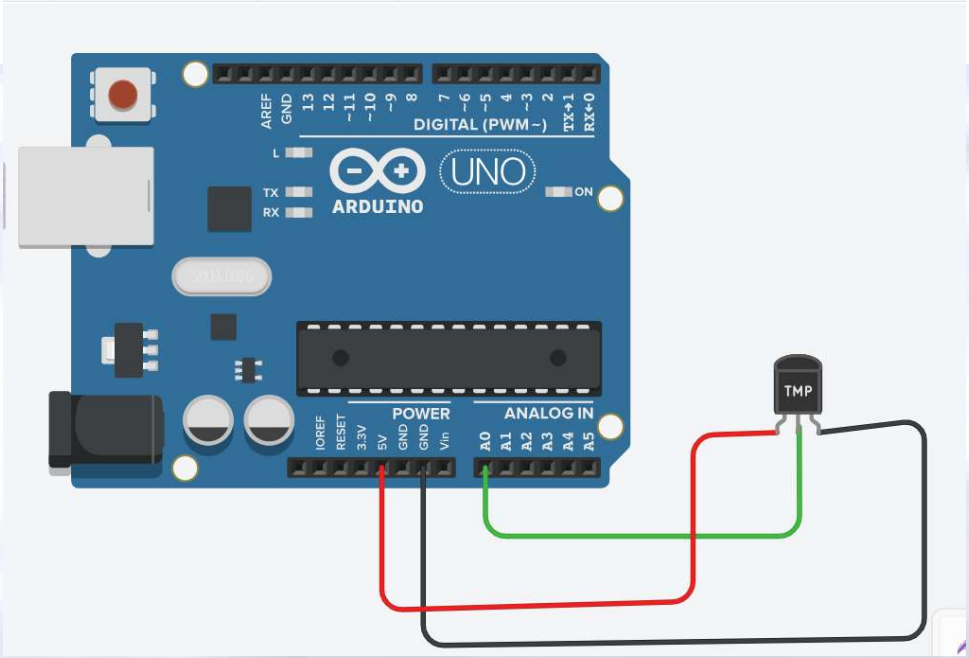
Materials:

- **Arduino board** (Uno, Nano, etc.)
- **LM35 temperature sensor** or **DHT11** temperature and humidity sensor
- **Breadboard** and **jumper wires**
- **USB cable** for connecting Arduino to your computer

Connecting the LM35 Temperature Sensor:

The LM35 has three pins:

- **VCC**: Connect to 5V on the Arduino.
- **GND**: Connect to GND on the Arduino.
- **Output**: Connect to an analog pin on the Arduino (e.g., A0).



Arduino Sketch: LM35 Temperature Sensor:

cpp Copy code

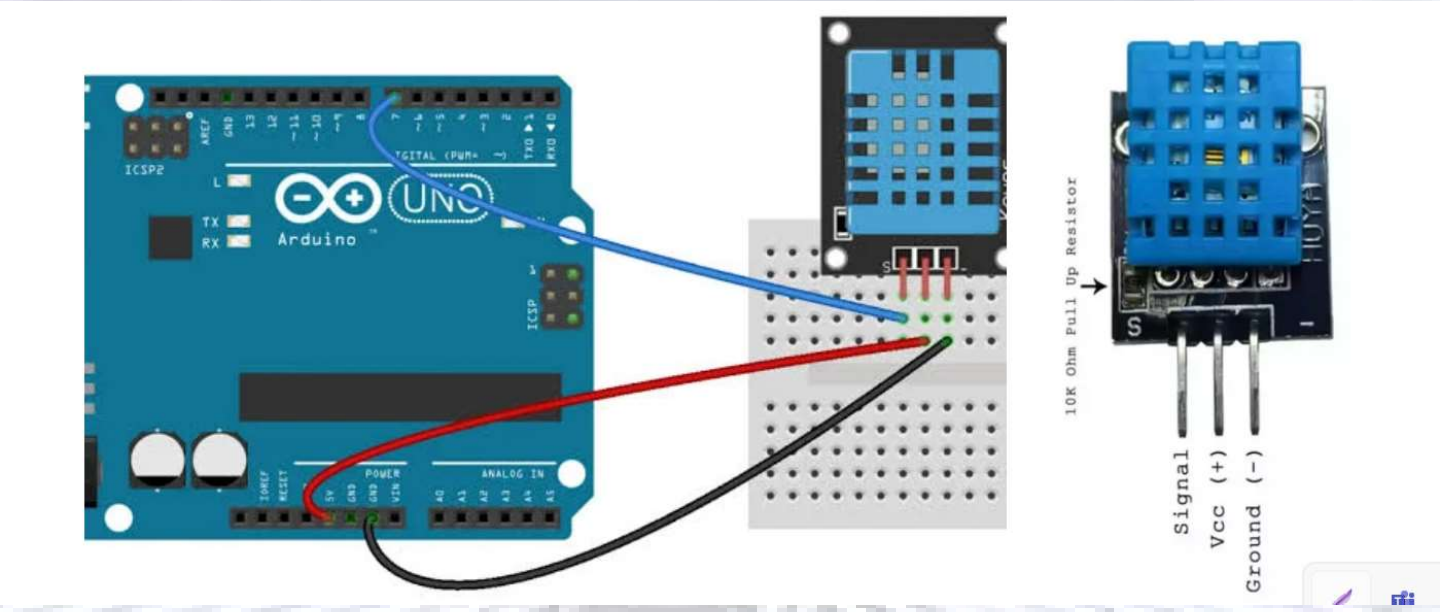
```cpp
int sensorPin = A0;  // Pin connected to the LM35's output
int sensorValue = 0; // Variable to store the sensor reading
float temperatureC = 0; // Variable to store the temperature in Celsius

void setup() {
  Serial.begin(9600);  // Start serial communication
}

void loop() {
  sensorValue = analogRead(sensorPin);  // Read the analog value from the sensor
  temperatureC = (sensorValue * 5.0 * 100.0) / 1024;  // Convert the analog value to tempe

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" °C");  // Print the temperature in Celsius

  delay(1000);  // Wait 1 second before taking the next reading
}
```

Connecting the DHT11 Temperature and Humidity Sensor:

The DHT11 has four pins, but only three are used:

- **VCC**: Connect to 5V on the Arduino.
- **GND**: Connect to GND on the Arduino.
- **Data**: Connect to a digital pin on the Arduino (e.g., D0,1,2,3,4,5, 6…).

You'll need the **DHT11 library** to interface with this sensor. Install it from the Arduino Library Manager.



Arduino Sketch: DHT11 Sensor:

cpp Copy code

```cpp
#include "DHT.h"

#define DHTPIN 2     // Pin connected to the data pin of DHT11
#define DHTTYPE DHT11   // DHT11 sensor type

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();  // Start the DHT sensor
}

void loop() {
  float temp = dht.readTemperature();  // Read temperature in Celsius
  float hum = dht.readHumidity();  // Read humidity

  // Check if any reads failed and exit early (to try again).
  if (isnan(temp) || isnan(hum)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.println(" °C");

  Serial.print("Humidity: ");
  Serial.print(hum);
  Serial.println(" %");

  delay(2000);  // Wait 2 seconds before taking the next reading
}
```

Summary:

- **Sensors** allow Arduino to interact with the physical world by detecting temperature, light, distance, and other parameters.
- **Analog sensors** provide continuous values, while **digital sensors** provide binary values.
- **Reading sensor data** is done using functions like analogRead() or digitalRead(), and the data can be displayed on the **Serial Monitor**.
- **Hands-on activity**: You can interface a **temperature sensor** like the LM35 or DHT11 with Arduino to measure and display temperature data in real time.

By the end of this lesson, you should be able to connect various sensors to an Arduino, read their data, and display the results using the Serial Monitor.